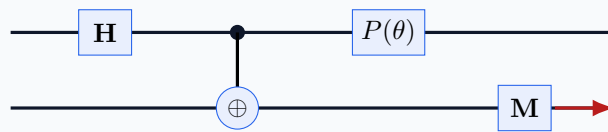


QPU Getting Started

Programming and Website Walkthrough



Bailey Forbes
September 13, 2026

A beginner-first introduction to the QPU website, programming vocabulary, classical logic, complete runnable protocols, and the first steps for correcting an error.

Contents

1	How to use the QPU website	1
1.1	Five programming words used in this guide	1
1.2	Interface words and how they relate	1
1.3	Run a bundled example without writing code	1
1.4	Compile a complete protocol	1
1.5	Complete examples versus fragments	2
1.6	When protocol checks report a problem	2
1.7	Saving and reopening work	2
2	First circuit: from source to result	2
2.1	Read the program line by line	2
2.2	What one run means	3
2.3	Adding user-controlled input	3
3	Classical computation foundations	3
3.1	Classical bits and logical state	3
3.2	Boolean algebra	3
3.3	Truth tables as complete finite specifications	4
3.4	Combinational logic and stored state	4
3.5	Why ordinary classical gates can lose information	4
3.6	Controls, targets, and workspace	5
3.7	Fan-out, copying, and the no-cloning boundary	5
3.8	Deterministic, probabilistic, and quantum models	5
3.9	Classical and QPU operation map	6
4	Worked example: reversible half-adder	6
4.1	Classical specification	6
4.2	QPU protocol	6
4.3	What every term is doing	6
4.4	Reversible state equation	7
4.5	Companion truth-table metadata	7
5	Worked example: randomness versus interference	7
5.1	Classical fair coin model	7
5.2	Quantum preparation with Hadamard	8
5.3	Insert phase and recombine paths	8
5.4	Term-by-term explanation	8
5.5	Comparison table	9
6	Where to continue	9

1 How to use the QPU website

This chapter assumes no programming experience. You can build circuits by clicking controls, or edit the text protocol directly. Both paths describe the same circuit.

1.1 Five programming words used in this guide

Source text The human-readable instructions in the protocol editor. It is ordinary text, like a note, with one QPU instruction per logical line.

Instruction One action for the QPU system, such as SET, H, CNOT, or MEASURE. The first word says what action to perform.

Argument Extra information after an instruction. In SET Q 0p, Q and 0p are arguments: they say which wire and which state.

Compile Translate source text into the ordered visual gates that the simulator can run. Compilation checks spelling, required inputs, and targets.

Run Apply the compiled gates to the selected starting state. Running changes simulator state; it does not rewrite the source text.

1.2 Interface words and how they relate

Word on screen or in source	Meaning
Particle / qubit	One simulated quantum two-state system. The interface may say particle where circuit theory says qubit.
Wire	The horizontal visual path followed by one qubit through the circuit.
Token	A source-text name, such as Q or Carry, that resolves to a wire.
Register	One or more qubit wires considered as a group.
Gate	A box or controlled symbol that transforms state.
Target	The wire changed by a gate.
Control	A wire whose state decides whether a controlled gate acts.
Protocol / process	The complete source description of a named circuit.

1.3 Run a bundled example without writing code

1. Open the navigation menu and choose **Circuit builder**.
2. Find **Load a starter circuit** and choose an example.
3. Inspect the horizontal wires and gate blocks on the circuit canvas.
4. Choose **Play Sequence** to watch gates advance, or **Run all** to jump to the final state.
5. Choose **Measure all** if the circuit has not already measured its outputs.
6. Read the particle/state display and runtime log. A measured qubit is a classical 0 or 1 outcome for that run.
7. Choose **Reset state** before repeating the same experiment.

1.4 Compile a complete protocol

Listings marked **Complete runnable protocol** may be copied as a whole:

Complete runnable protocol

```
MAIN-PROCESS FirstWebsiteCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

1. Scroll to **Compile text into a circuit**.
2. Replace the editor contents with the complete listing.
3. Choose **Compile protocol**.
4. Read **Protocol checks**. Zero errors means compilation may proceed. Warnings identify accepted syntax that may not behave as expected.
5. Return to the run controls and choose **Run all**.

Expected result: the H gate prepares equal 0 and 1 amplitudes. MEASURE returns either 0 or 1. Repeating from Reset state should produce both outcomes over many runs.

1.5 Complete examples versus fragments

This documentation uses three listing labels:

Complete runnable protocol Copy the entire block into the protocol editor and compile it.

Instruction fragment A few lines demonstrating one syntax rule. Add them to a complete process rather than compiling the fragment alone.

Companion metadata Truth-table text saved separately as `.qpuio`; do not paste it into the protocol editor.

1.6 When protocol checks report a problem

An **error** blocks successful compilation. A **warning** means the text is accepted but may use inactive, incomplete, or surprising behavior. Each diagnostic gives a line number, the relevant source line when available, and a suggested correction.

Choose **Open Correction Lab for guided help** below the diagnostic list when the correction is not obvious. The Correction Lab can test a protocol, probe outputs, explain failures, and propose gate-level guidance. It does not change protected bundled truth tables.

1.7 Saving and reopening work

Choose **Download as .qpucir** to save protocol source. The `-qpucir.txt` option contains the same source for devices that only handle plain text files. Use the navigation menu's **Upload files** page to reopen a protocol, optionally together with its matching `.qpuio` truth-table metadata.

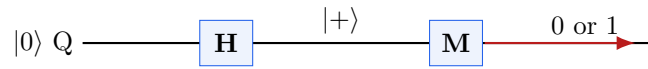
2 First circuit: from source to result

The shortest useful learning path is to prepare one qubit, transform it, measure it, and declare it as an output:

```
MAIN-PROCESS FirstCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

2.1 Read the program line by line

1. `MAIN-PROCESS FirstCircuit` gives the protocol a name.
2. `CREATETOKEN -I Q` reserves a logical wire named Q.
3. `SET Q 0p` prepares Q in the basis state $|0\rangle$.
4. `H -I Q -0 Q` applies Hadamard to that same wire, producing $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
5. `MEASURE -I Q` samples 0 or 1 and collapses the state.
6. `RETURNVALS Q` declares Q as the first public result.



2.2 What one run means

One execution gives one random-looking sample. For this circuit, theory predicts equal probability for 0 and 1. Getting 0 once does not mean H failed, and getting 1 once does not prove a bias. Repeat the circuit to compare observed frequencies with the predicted distribution.

2.3 Adding user-controlled input

Move a wire into PARAMS when callers or truth-table rows should choose its start state:

```
PARAMS: Input:state

MAIN-PROCESS Inverter
X -I Input -0 Input
MEASURE -I Input
RETURNVALS Input
```

The matching deterministic truth table is:

Input	Output
0p	1p
1p	0p

The rest of this manual explains every symbol in these examples before moving to multi-qubit gates, child processes, and full truth-table files.

3 Classical computation foundations

Quantum circuits build on ideas from classical digital logic. Understanding bits, Boolean functions, truth tables, and reversible computation makes the QPU gate model much easier to use.

3.1 Classical bits and logical state

A classical bit has one definite logical value:

$$b \in \{0, 1\}.$$

Physical computers may represent that value with voltage, charge, magnetic orientation, or another measurable quantity. Digital logic ignores most physical detail and works with the abstraction 0 or 1.

A wire carries a bit from one operation to another. A *gate* computes a function of one or more incoming bits. For example, NOT has one input and one output:

$$\text{NOT}(0) = 1, \quad \text{NOT}(1) = 0.$$

In QPU source, a token names the logical wire. On computational-basis states, `X -I Q -0 Q` has the same measured input/output relationship as classical NOT.

3.2 Boolean algebra

Boolean algebra describes functions whose values are 0 and 1. This guide uses:

Operation	Symbol	Spoken form	Meaning
NOT	$\neg A$	not A	1 when A is 0; 0 when A is 1.
AND	$A \wedge B$	A and B	1 only when both inputs are 1.
OR	$A \vee B$	A or B	1 when either or both inputs are 1.
XOR	$A \oplus B$	A exclusive-or B	1 when exactly one input is 1.
NAND	$\neg(A \wedge B)$	not-and	The complement of AND.

For bits, XOR is addition modulo two:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

There is no carry in modulo-two addition. Ordinary binary addition handles the carry separately, which is why a half-adder produces both Sum and Carry.

3.3 Truth tables as complete finite specifications

A truth table lists a function's output for every possible input pattern. With n binary inputs there are 2^n patterns. AND, OR, XOR, and NAND can be compared in one table:

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$	$\neg(A \wedge B)$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

For a deterministic combinational circuit, this table completely specifies the classical input/output behavior. It does not reveal the circuit's internal gate arrangement, propagation time, power consumption, or quantum phase.

3.4 Combinational logic and stored state

A *combinational* output depends only on current inputs. Adders and the Boolean gates above are combinational. A *sequential* circuit also depends on stored state from an earlier step, as in counters and latches.

QPU cycle suffixes can document successive stages, and INCREASECYCLE marks a workspace boundary, but the current compiled circuit is still one ordered gate sequence. Accepted `SAVE_STATE` and `LOAD_STATE` syntax does not yet implement a classical memory checkpoint. When modeling stateful behavior, expose previous state explicitly as `PARAMS` and return next state explicitly through `RETURNVALS`.

3.5 Why ordinary classical gates can lose information

A function is reversible only if each output identifies exactly one input. Classical NOT is reversible: seeing output 1 proves the input was 0. AND is not: output 0 could have come from 00, 01, or 10.

$$(A, B) \xrightarrow{\text{AND}} A \wedge B$$

maps four input patterns onto only two output values, discarding information. Quantum evolution through closed-system gates must be unitary and therefore reversible. A quantum/reversible AND preserves both inputs and mutates another target:

$$(A, B, t) \mapsto (A, B, t \oplus (A \wedge B)).$$

Knowing the output triple lets the transformation be run backward to recover the input triple.

3.6 Controls, targets, and workspace

These terms connect classical logic to QPU syntax:

Control A wire whose value decides whether a target operation occurs. In `CNOT -I A -0 T`, A is the control.

Target The wire changed by an operation. T is the target in that `CNOT`.

Workspace or ancilla An extra wire introduced to hold an intermediate or output while preserving inputs. It is commonly initialized to $|0\rangle$.

Garbage Intermediate information left in workspace after the desired answer has been computed. Reversible algorithms often *uncompute* temporary gates to return garbage wires to zero.

Uncomputation Applying inverse operations in reverse order to clean workspace without erasing information irreversibly.

The QPU language calls logical wire names tokens. `CREATETOKEN` reserves workspace, `SET` prepares it, `-I` lists inputs/controls, and `-0` names targets.

3.7 Fan-out, copying, and the no-cloning boundary

Classical logic freely copies a bit onto several wires. If the source is a known basis bit and targets begin at zero, `CNOT` reproduces that basis value:

$$|x\rangle|0\rangle \xrightarrow{\text{CNOT}} |x\rangle|x\rangle, \quad x \in \{0, 1\}.$$

For a superposition $\alpha|0\rangle + \beta|1\rangle$, the same circuit produces

$$\alpha|00\rangle + \beta|11\rangle,$$

not

$$(\alpha|0\rangle + \beta|1\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle).$$

The first state is generally entangled; the second would be two independent copies and contains cross terms. The no-cloning theorem forbids a universal gate that copies every unknown quantum state. QPU aliases similarly give two names to one wire; they do not create a physical copy.

3.8 Deterministic, probabilistic, and quantum models

Model	State description	Transformation	Observation
Deterministic classical	One definite bit string	Boolean function	The definite output bits
Probabilistic classical	Probabilities over bit strings	Stochastic update	A sampled string
Pure-state quantum	Complex amplitudes over basis strings	Unitary matrix	A sampled string after magnitude-squared probabilities and collapse

Negative and imaginary amplitudes are not negative or imaginary probabilities. They carry phase. Two computational paths can add constructively or cancel destructively before probabilities are calculated. This interference has no counterpart in an ordinary classical random-choice table.

3.9 Classical and QPU operation map

Classical idea	QPU operation	Important difference
NOT	X or NOT	Acts linearly on superpositions; X is unitary.
XOR into target	CNOT	Preserves the control and can entangle.
AND into target	CCNOT or AND	Preserves controls; target is XOR-mutated.
Swap variables	SWAP	Exchanges complete quantum wire states.
Random bit	H then MEASURE	H first creates coherent amplitudes, not an ordinary hidden random choice.
Read a bit	MEASURE	Sampling collapses a superposition and can disturb entanglement.
Initialize zero	SET token 0p	Compiles to state preparation/reset behavior.
Function output	RETURNVALS	Declares an interface; does not itself copy or measure a wire.

4 Worked example: reversible half-adder

A half-adder adds two one-bit numbers without a carry input. Classical theory defines two outputs:

$$\text{Sum} = A \oplus B, \quad \text{Carry} = A \wedge B.$$

The name *half-adder* means it handles half of the full-adder interface: there is no C_{in} input.

4.1 Classical specification

A	B	Carry	Sum	Binary interpretation
0	0	0	0	$0 + 0 = 00_2$
0	1	0	1	$0 + 1 = 01_2$
1	0	0	1	$1 + 0 = 01_2$
1	1	1	0	$1 + 1 = 10_2$

The Carry column is the high-order bit and Sum is the low-order bit. In the last row, decimal $1 + 1 = 2$, written 10_2 in binary.

4.2 QPU protocol

```

PARAMS: A:state B:state

MAIN-PROCESS HalfAdder
CREATETOKEN -I Carry Sum
SET Carry 0p
SET Sum 0p

XOR -I A B -O Sum
AND -I A B -O Carry

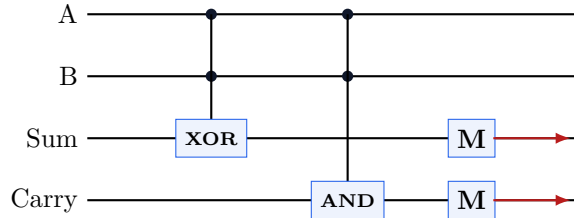
MEASURE -I Carry
MEASURE -I Sum
RETURNVALS Carry Sum

```

4.3 What every term is doing

1. **PARAMS** declares A and B as caller-controlled state inputs. In a basis-state truth-table test they receive 0p or 1p.

2. **Carry and Sum are workspace targets.** CREATETOKEN reserves their names; SET prepares both in zero.
3. **XOR uses A and B as controls/inputs.** It mutates Sum according to $s' = s \oplus A \oplus B$. Because $s = 0$, the result is $A \oplus B$.
4. **AND uses the same preserved controls.** It mutates Carry according to $c' = c \oplus (A \wedge B)$. Because $c = 0$, Carry becomes the classical carry predicate.
5. **MEASURE produces classical samples.** For basis inputs this example is deterministic. If A or B is in superposition, outputs may be entangled with the inputs and measurement samples a branch.
6. **RETURNVALS defines interface order.** Carry is output column zero and Sum is output column one. It does not perform measurement itself.



4.4 Reversible state equation

Before measurement, the full basis-state action is

$$|A, B, s, c\rangle \mapsto |A, B, s \oplus A \oplus B, c \oplus (A \wedge B)\rangle.$$

The familiar half-adder is the special case $s = c = 0$. Writing the complete equation avoids a common mistake: XOR and AND do not overwrite arbitrary targets. They reversibly combine results with the targets' previous states.

4.5 Companion truth-table metadata

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

This file compares the measured QPU outputs with the classical half-adder specification. Passing all four rows demonstrates basis-state equivalence; it does not by itself characterize phases for superposition inputs.

5 Worked example: randomness versus interference

Classical random choice and quantum superposition can produce identical single-measurement frequencies. They are not the same theory. This example uses an intermediate phase change to distinguish them.

5.1 Classical fair coin model

Imagine a classical variable R chosen by a fair coin:

$$P(R = 0) = \frac{1}{2}, \quad P(R = 1) = \frac{1}{2}.$$

Once chosen, R is one definite value even if the observer has not looked. Applying classical NOT changes 0 to 1 and 1 to 0, but the distribution remains half and half. There is no operation that makes the probability of one hidden branch cancel the other branch.

5.2 Quantum preparation with Hadamard

```
MAIN-PROCESS QuantumCoin
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Before measurement,

$$|0\rangle \xrightarrow{H} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

The amplitudes are $1/\sqrt{2}$ and $1/\sqrt{2}$; squaring magnitudes gives the same half-and-half distribution as the classical coin. If measurement happens now, this experiment alone cannot distinguish the two descriptions.

5.3 Insert phase and recombine paths

```
MAIN-PROCESS PhaseInterference
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
Z -I Q -0 Q
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Track the state:

$$|0\rangle \xrightarrow{H} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{Z} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \xrightarrow{H} |1\rangle.$$

5.4 Term-by-term explanation

Superposition After the first H, both basis states have coherent amplitudes. No measurement has selected one.

Phase Z multiplies only the $|1\rangle$ amplitude by $-1 = e^{i\pi}$. Its magnitude remains $1/\sqrt{2}$, so immediate 0/1 probabilities remain equal.

Relative phase The plus relationship between amplitudes becomes a minus relationship. A global minus on both terms would not matter; changing only one term does.

Interference The final H mixes the basis amplitudes. Contributions to the $|0\rangle$ output cancel, while contributions to $|1\rangle$ add.

Measurement The final state is $|1\rangle$, so measurement returns 1 with probability one.

The cancellation can be seen directly from matrix multiplication:

$$H \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Classical probability weights are nonnegative and would add rather than cancel. Quantum amplitudes are signed or complex, so phase can create destructive and constructive interference before the Born rule turns amplitudes into probabilities.

5.5 Comparison table

Question	Classical fair coin	H–Z–H quantum circuit
State before observation	One hidden definite bit sampled fairly	Coherent complex amplitude vector
Effect of middle sign/phase	No probability analogue that preserves a fair distribution and later cancels a branch	Z changes relative phase without changing immediate basis probabilities
Can paths cancel?	No; probabilities add	Yes; amplitudes add and may cancel
Final result in this example	Still a distribution unless another classical rule forces a value	Deterministic 1

6 Where to continue

- Read **QPU Theory Guide** when amplitude, phase, or matrix notation is unfamiliar.
- Open **QPU Language Reference** while writing exact syntax.
- Open **QPU Examples and Troubleshooting** when Protocol checks reports an error or a truth-table test fails.